0. There are three houses, three nationals, three animals, three cigarettes.
1. The Englishman lives in the first house on the left.
2. In the house immediately on the right of that housing the wolf, they smoke Luky Strike.
3. The Spaniard smokes Kent.
4. The Russian has a horse.

Goals: Who smokes LM? Who has the dog?

There are two solutions:

I.	**Englishman**	**dog**	**LM**
	Spaniard	**wolf**	**Kent**
	Russian	**horse**	**LS**

II.	**Englishman**	**wolf**	**LM**
	Russian	**horse**	**LS**
	Spaniard	**dog**	**Kent**

list = symbol*
pos(list, list, list, symbol, symbol, symbol, symbol)
solve(list, list, list)
delete(symbol, list, list)
perm(list, list)
candidate(list, list, list)
constraints(list, list, list)
posleft(symbol, symbol)

```
solve(N, A, C) :-
        candidate(N, A, C),
        constraints(N, A, C),
        nl.

candidate(N, A, C) :-
        perm([eng,spa,rus], N),
        perm([dog,wol,hor], A),
        perm([lm,ken,ls], C).

constraints(N, A, C) :-
        pos(N, A, C, eng, _, _, p1),
        pos(N, A, C, _, wol, _, H1),
        posleft(H1, H2),
        pos(N, A, C, _, _, ls, H2),
        pos(N, A, C, spa, _, ken, _),
        pos(N, A, C, rus, hor, _, _).

pos([N1,_,_], [A1,_,_], [C1,_,_], N1, A1, C1, p1).
pos([_,N2,_], [_,A2,_], [_,C2,_], N2, A2, C2, p2).
pos([_,_,N3], [_,_,A3], [_,_,C3], N3, A3, C3, p3).

perm([], []).
perm(Y, [A|X]) :-
        delete(A, Y, Y1),
        perm(Y1, X).

delete(A, [A|X], X).
delete(A, [B|X], [B|Y]) :-
        delete(A, X, Y).

posleft(p1, p2).
posleft(p2, p3).
```

This is a classical problem in computer science. The objective is to place eight queens on a chessboard so that no two queens are attacking each other; i.e., no two queens are in the same row, the same column, or on the same diagonal. We generalize this original problem by allowing for an arbitrary dimension N of the chessboard.

We represent the positions of the queens as a list of numbers 1..N. Example: [4,2,7,3,6,8,5,1] means that the queen in the first column is in row 4, the queen in the second column is in row 2, etc. By using the permutations of the numbers 1..N we guarantee that no two queens are in the same row. The only test that remains to be made is the diagonal test. A queen placed at column X and row Y occupies two diagonals: one of them, with number C = X-Y, goes from bottom-left to top-right, the other one, numbered D = X+Y, goes from top-left to bottom-right. In the test predicate we keep track of the already occupied diagonals in Cs and Ds.

% The first version is a simple generate-and-test solution.

list = integer*
queens_1(integer,list)
range(integer, integer, list)
perm(list, list)
delete(integer, list, list)
test(list)
test(list, integer, list, list)
member(integer, list)
queens_2(integer,list)
perm_test(list, list, integer, list, list)

```
% queens_1(N,Qs) :- Qs is a solution of the N-queens problem
queens_1(N,Qs) :-
        range(1,N,Rs),
        perm(Rs,Qs),
        test(Qs).

% range(A,B,L) :- L is the list of numbers A..B
range(A,A,[A]).
range(A,B,[A|L]) :-
        A < B,
        A1 is A+1,
        range(A1,B,L).

% perm(Xs,Zs) :- the list Zs is a permutation of the list Xs
perm([],[]).
perm(Qs,[Y|Ys]) :-
        delete(Y,Qs,Rs),
        perm(Rs,Ys).

delete(X,[X|Xs],Xs).
delete(X,[Y|Ys],[Y|Zs]) :-
        delete(X,Ys,Zs).

% test(Qs) :- the list Qs represents a non-attacking queens solution
test(Qs) :-
        test(Qs,1,[],[]).

% test(Qs,X,Cs,Ds) :- the queens in Qs, representing columns X to N,
% are not in conflict with the diagonals Cs and Ds
test([],_,_,_).
test([Y|Ys],X,Cs,Ds) :-
        C is X-Y,
        not(member(C,Cs)),
        D is X+Y,
        not(member(D,Ds)),
        X1 is X + 1,
```

```
        test(Ys,X1,[C|Cs],[D|Ds]).

% member(E, L) :- the element E is a member of the list L
member(E, [E|_]).
member(E, [_|L]) :-
        member(E, L).


% Now, in version 2, the tester is pushed completely inside the
% generator permu.

queens_2(N,Qs) :-
        range(1,N,Rs),
        perm_test(Rs,Qs,1,[],[]).

perm_test([],[],_,_,_).
perm_test(Qs,[Y|Ys],X,Cs,Ds) :-
        delete(Y,Qs,Rs),
        C is X-Y,
        not(member(C,Cs)),
        D is X+Y,
        not(member(D,Ds)),
        X1 is X+1,
        perm_test(Rs,Ys,X1,[C|Cs],[D|Ds]).
```

How can a knight jump on an NxN chessboard in such a way that it visits every square exactly once?

pos = p(integer, integer)
list = pos*
closed_knights(integer, list)
knights(integer, list)
knights(integer, integer, list, list)
member(pos, list)
jump(integer, pos, pos)
jump_dist(integer, integer)

```
% closed_knights(N,Knights) :- Knights is a knight's tour on a NxN
% chessboard which ends at the same square where it begun.

closed_knights(N,Knights) :-
        knights(N,Knights),
        Knights = [p(X,Y)|_],
        jump(N,p(X,Y),p(1,1)).

% knights(N,Knights) :- Knights is a knight's tour on a NxN chessboard

knights(N,Knights) :-
        M is N*N-1,
        knights(N,M,[p(1,1)],Knights).

% knights(N,M,Visited,Knights) :- the list of squares Visited must be
% extended by M further squares to give the solution Knights of the
% NxN chessboard knight's tour problem.

knights(_,0,Knights,Knights).
knights(N,M,Visited,Knights) :-
        Visited = [p(X,Y)|_],
        jump(N,p(X,Y),p(U,V)),
        not(member(p(U,V),Visited)),
        M1 is M-1,
```

```
        knights(N,M1,[p(U,V)|Visited],Knights).

% member(E, L) :- the element E is a member of the list L

member(E, [E|_]).
member(E, [_|L]) :-
        member(E, L).

% jumps on an NxN chessboard from square A/B to C/D

jump(N,p(A,B),p(C,D)) :-
        jump_dist(X,Y),
        C is A+X, C > 0, C <= N,
        D is B+Y, D > 0, D <= N.

% jump distances

jump_dist(1,2).
jump_dist(2,1).
jump_dist(2,-1).
jump_dist(1,-2).
jump_dist(-1,-2).
jump_dist(-2,-1).
jump_dist(-2,1).
jump_dist(-1,2).
```

Paleo-arithmetics: SEND + MORE = MONEY

```
list = integer*
solve(list)
domain(list, integer, integer)
select(integer, integer, integer)
different(list)
sum(list, list, list)
sum(list, list, list, integer)
inc(list, list)
reverse(list, list)
append(list, integer, list)
member(integer, list)

%solve([S,E,N,D,M,O,R,Y]) :-                % SEND + MORE = MONEY
%       domain([S,E,N,D,M,O,R,Y], 0, 9),
%       S>0, M>0,
%       different([S,E,N,D,M,O,R,Y]),
%       sum([S,E,N,D], [M,O,R,E], [M,O,N,E,Y]).

solve([A,B,C,D]) :-                        % A + B = CD
        domain([A,B,C,D], 0, 9),
        A>0, C>0,
        different([A,B,C,D]),
        sum([A], [B], [C,D]).

%************************

domain([], _, _).
domain([H|T], M, N) :-
        select(H, M, N),
        domain(T, M, N).

select(M, M, N) :-
        M <= N.
select(H, M, N) :-
```

```
        M <= N,
        M1 is M + 1,
        select(H, M1, N).

different([]).
different([H|T]) :-
        not(member(H,T)),
        different(T).

%************************

sum(N1, N2, NR) :-
        reverse(N1, L1),
        reverse(N2, L2),
        sum(L1, L2, LR, 0),
        reverse(LR, NR).

sum([], L, L, 0) :- !.
sum([], L, NewL, 1) :- !,
        inc(L, NewL).
sum(L, [], L, 0) :- !.
sum(L, [], NewL, 1) :- !,
        inc(L, NewL).

sum([H1|L1], [H2|L2], [H|LR], Count) :-
        H = (H1 + H2 + Count) mod 10,
        NewCount = (H1 + H2 + Count) div 10,
        sum(L1, L2, LR, NewCount).

inc([], [1]).
inc([H|L], [NewH|L]) :-
        H < 9,
        !,
        NewH = H+1.
inc([9|L], [0|NewL]) :-
        inc(L, NewL).
```

```
%************************

reverse([], []).
reverse([H|T], L) :-
        reverse(T, L1),
        append(L1, H, L).

append([], A, [A]).
append([H|T], A, [H|L]) :-
        append(T, A, L).

member(E, [E|_]).
member(E, [_|T]) :-
        member(E, T).
```